EnduroSense study guide

Everything in the project, top to bottom, followed by the questions a panel is likely to ask, with follow-ups.

How to use this guide

1. Read Part A once, in order. It is the whole project as a story.
2. Learn the numbers in Part B by heart. There are not many.
3. Work through Part C (the questions) out loud. Cover the answer, say yours, then compare.
4. Read Part D the day before. It is the list of things you must not over-claim.

Every number here is the test-set result unless it says "development".

Part A. The project, from top to bottom

A1. The problem and the idea

The brief. Build a machine-learning model that predicts the remaining flight time of a UAV battery, and compare five algorithms: Linear Regression, Random Forest, XGBoost, LSTM and GRU.

Why "minutes left" is not enough for a real decision.

- **Minutes depend on what the drone is doing right now.** Minutes left = energy left ÷ power being drawn now. A drone that is hovering or descending draws little power, so it shows many minutes, whatever the next mission needs. Our own results show this: models whose energy error ranged from 2.6 to 4.5 Wh all had about the same minutes error (0.87 to 1.02 min). The minutes error is dominated by not knowing what the drone will do next, not by the battery estimate.
- **A single number hides how sure it is.** Two batteries can show the same estimate while one is much less certain. A safety decision needs to know that.

The idea. Answer the question a scheduler actually has: *can this battery fly this mission and still keep a safe reserve?*

Piece	What it predicts	From what
Model A	Energy available: watt-hours (Wh) the battery can still give above the reserve	Battery telemetry
Model B	Energy required: Wh the planned mission will use	The mission plan
Ranges	How sure each prediction is	The models' errors on unseen data
P(success)	The chance that energy available ≥ energy required	The two ranges
Decision	GO only if P(success) ≥ τ (0.95)	

Why energy and not minutes. Energy adds up. A mission's energy is the sum of its parts (climb + legs + hover + descent), and the battery's energy drops by exactly what was used. Minutes do not add up that way.

The reserve. 22.6 V measured at rest (motors off), about 3.77 V per cell for this 6-cell pack. "Energy available" always means energy *above* that reserve, so a successful mission ends with the reserve intact. An "unsafe" mission is one that cuts into the reserve. It does not mean a crash.

Status of the settings. The reserve (22.6 V), the threshold τ (0.95) and the range level (90%) are defaults in one file, `config.yaml`. The mentor has not yet confirmed them. Changing any of them takes a rerun of about 30 minutes.

The whole pipeline in one picture

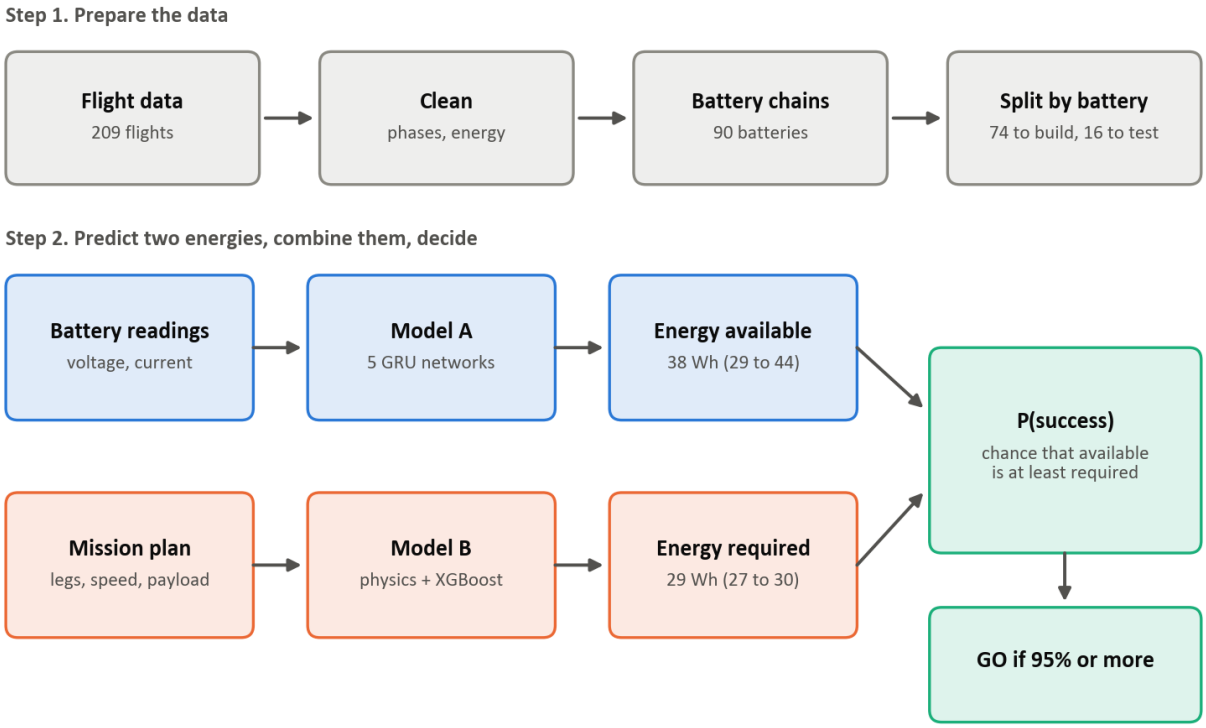


Figure 1. The pipeline. The example numbers are the worked example of section A12.

1. **Prepare the data.** Clean the recordings, work out which flights shared a battery, and set 16 batteries aside for the final test.
2. **Model A** reads the battery and gives the energy available, with a range.
3. **Model B** reads the mission plan and gives the energy required, with a range.
4. **P(success)** is the chance that the first is at least the second.
5. **Decision:** GO at 95% or more.

A2. The data

Source. The public Carnegie Mellon University dataset "Data Collected with Package Delivery Quadcopter Drone" (Rodrigues et al., 2021).

Fact	Value
Drone	DJI Matrice 100
Battery	DJI TB47D, 6 cells, 4.5 Ah × 22.2 V = 99.9 Wh
Flights	209, over 18 flying days (April to October 2019)
Sensor readings	257,896, at about 8 per second
Sensors	Battery voltage and current, wind (an anemometer on the drone), GPS position, speed and orientation
Flight programme	5 speeds (4, 6, 8, 10, 12 m/s) × 3 payloads (0, 250, 500 g) × 4 altitudes (25, 50, 75, 100 m)
Routes	R1 for 182 flights (a closed loop of 3 legs: 140, 198 and 119 m). A few flights on R2 to R7, plus hover and ground tests
A typical flight	About 3 minutes, about 21 Wh

Why this dataset. It is real flight data, it records voltage and current (so energy can be computed exactly), and it varies speed, payload and altitude in a planned grid.

Its weakness. One drone, one battery type, almost all flights on one route, and short flights. Nobody recorded which physical battery was used on each flight. We had to work that out (section A4).

What we found wrong in the data and fixed

Problem	How we found it	Fix
Flight start times are stored as text ("9:22"), and text sorting puts "9:22" after "10:05". Seven days were in the wrong order	A verification pass: battery chains looked broken	Always order by the real timestamp. A test checks single-digit hours
4 start times were mis-recorded (for example 22:31 instead of 10:31)	Voltage did not continue between neighbouring flights	Corrected in code from a small file, each with voltage evidence. The raw data is never edited
The documentation says vertical speed is positive downwards. The data shows positive is upwards	Correlation with the change in altitude is +0.77	Handled with the correct sign
The altitude reading drifts by up to 14 m after landing	31 flights ended "hovering" at height with zero current	Rule: if current is under 2 A, the motors are off and the drone is on the ground
76 sensor glitches (22 voltage, 54 current; 0.03% of readings)	Jumps that no change in load explains	Replaced by the median of the 5 surrounding readings and flagged

A3. Preparing the data (Phase 1)

Cleaning. Check for missing values, time going backwards, values outside physical limits, and gaps. There were 10 recording gaps longer than 1 second (the largest 5.3 s). Energy is still counted across them.

Flight phases. Every reading is labelled ground, climb, cruise, hover or descent:

- **Ground:** below 1.5 m, or current under 2 A.
- **Cruise:** horizontal speed at least 1.5 m/s.
- **Climb / descent:** vertical speed above 0.5 m/s in size.
- **Hover:** in the air and none of the above.
- Fragments shorter than 1 second are merged into their neighbour.

195 of the 196 route flights follow the expected order: ground → climb → cruise → descent → ground. Climbing takes about 17% of a flight's energy, cruising 50%, descending 22%.

Energy. Power = voltage × current. Energy = power × time, summed reading by reading. This gives the energy used at every instant, per flight and per battery.

Wind. The anemometer is on the drone, so it measures wind plus the drone's own motion. We take the "ambient wind" from moments when the drone is in the air, above 10 m, and not moving sideways.

A4. Battery chains and the locked test set

Why chains matter. A battery that is used for three flights in a row gives three recordings. If one of those flights went into the training data and another into the test data, the model would be tested on a battery it had already seen. Its results would look better than they are. This is called data leakage.

How we rebuilt them. A battery at rest (motors off) shows its true charge as a voltage. Between two flights on the same battery, that resting voltage can only stay the same or recover slightly. So:

- Take the resting voltage at the end of one flight and at the start of the next flight the same day.

- If the jump is between −0.10 V and +0.40 V, it is the same battery. A clear drop, or a big jump up, means a different battery was put in.
- The window is deliberately lopsided: a resting battery recovers, it does not lose voltage.

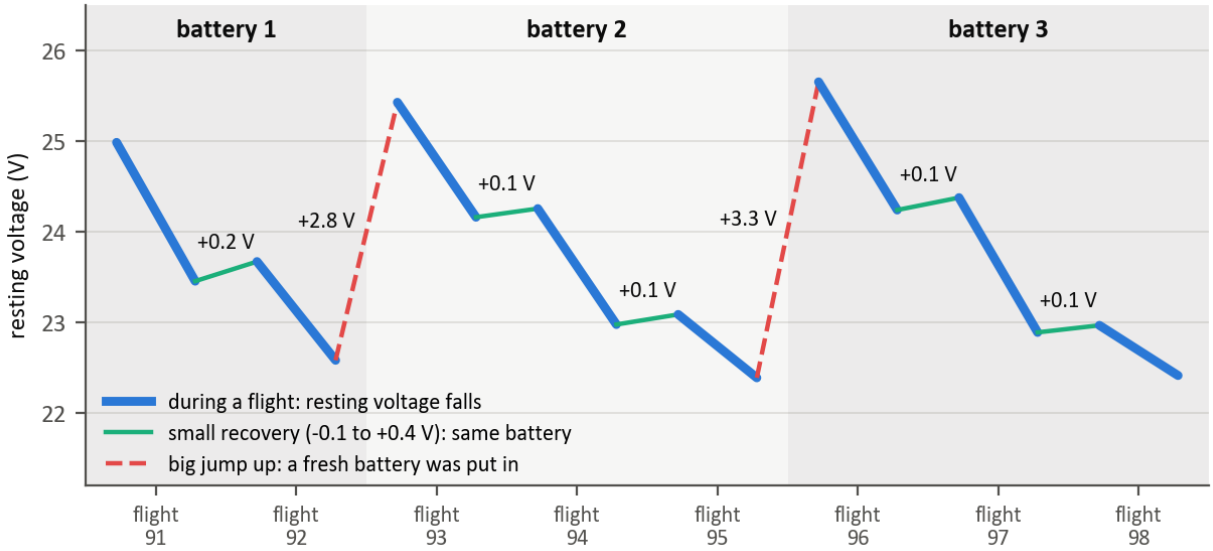


Figure 2. Eight flights flown one after another on one day. Small recoveries link flights into one battery; a big jump up means a fresh battery.

Result. 90 battery chains. 71 of them have more than one flight (covering 190 flights). 6 links are borderline and are flagged. We checked how sensitive this is: other reasonable windows give 87 to 92 chains.

The locked test set.

- **The unit is the chain,** never the single reading or flight.
- **16 chains (40 flights) are the test set.** They were chosen before any model was built.
- **How they were chosen:** 5,000 random candidate splits were scored on how well the test part matches the whole data (share of flights, mix of speeds, payloads and altitudes, share of chains reaching the reserve). The best-balanced one was kept. The longest route, R6, was forced into the test set so that the final evaluation includes a distance never seen in training.
- **The lock:** the split file carries a hash, so any edit is detected. The code refuses to load test rows unless the final evaluation asks for them, and every opening is written to a log.
- **The other 74 chains (169 flights)** are the development data. They are divided into 5 folds, again by chain, for cross-validation.

Cross-validation, in one sentence. Train on 4 folds, predict the 5th, repeat 5 times, so every battery is predicted by a model that never saw it.

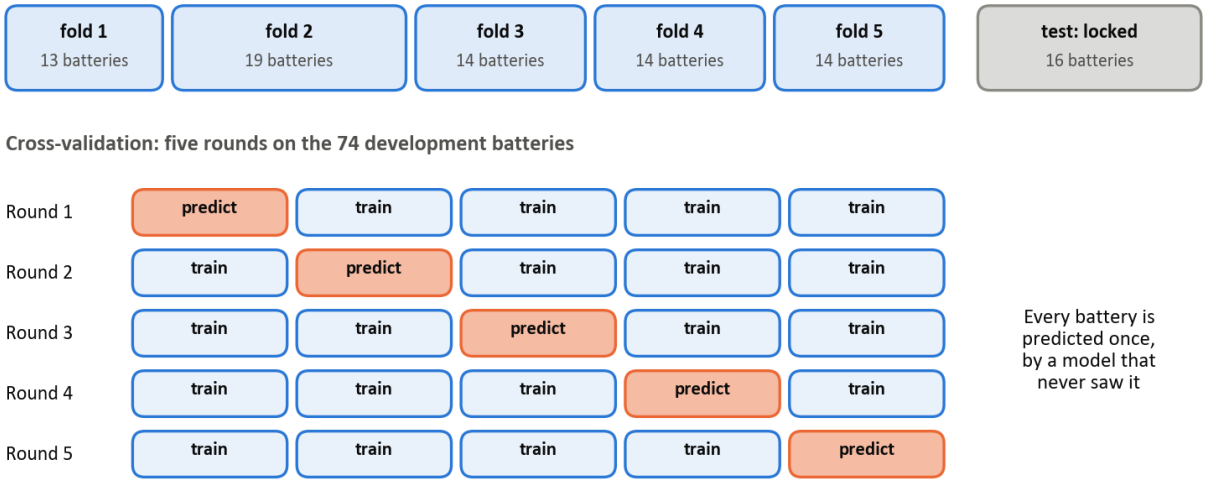


Figure 3. The split. The test batteries stay locked. The development batteries take turns at being predicted.

A5. What Model A learns (Phase 2)

The label: "energy left above the reserve"

For a battery that was flown down to the reserve, we know how much energy was drawn before its resting voltage reached 22.6 V. Call that E_reserve. Then at any moment:

remaining energy = E_reserve − energy drawn so far

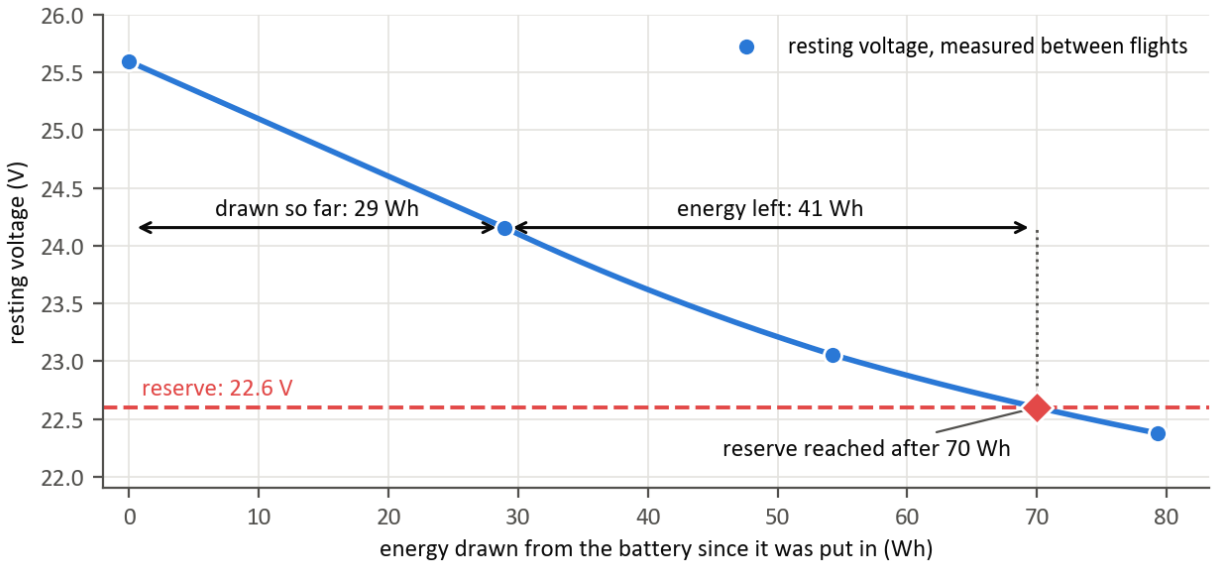


Figure 4. The label for one development battery. The dots are resting voltages measured between flights.

Reading the picture. This battery was put in at 25.6 V. Its resting voltage reached the 22.6 V reserve after 70 Wh had been drawn, so E_reserve = 70 Wh. After its first flight it had given 29 Wh, so the label at that moment is 70 − 29 = 41 Wh. During its third flight the label passes zero: from then on the battery is inside the reserve.

The complication. We only know the resting voltage between flights, not continuously. So the exact point where the battery crossed 22.6 V lies between two resting measurements and has to be located.

- **A straight line between the two points was wrong by +3.3 Wh on average,** because the discharge curve is not straight: the battery loses 0.054 V per Wh near full but only 0.029 V per Wh near the reserve.
- **So we fitted the curve** (how fast voltage falls per Wh, as a function of voltage) on development batteries only, and located the crossing along it. Bias: −0.04 Wh. Spread: 1.5 Wh.
- **Chains that stop just above the reserve** (within 0.3 V) are extended along the same curve. These labels are noisier (about 2.2 Wh).

What this means. The "truth" itself is uncertain by about 1.5 to 2.2 Wh. No model can be shown to be more accurate than that.

How many labels. 60 of the 90 chains can be labelled (31 measured, 29 extended). That gives 29,592 one-second rows; 23,884 are development data.

What the batteries look like. Batteries that start full give 64.7 ± 7.3 Wh before the reserve (47 to 77 Wh). That spread between batteries is what Model A has to learn. A model that assumed every battery was the same would be off by 13.6 Wh.

The inputs: 25 features, all using only the past

Group	Features
Right now	voltage, current, power
Last 10 s and 30 s	average, spread and trend of voltage and current; average power
Battery health	internal resistance (how much the voltage drops per amp, over 30 s); voltage sag below the resting voltage
Charge when inserted	resting voltage before this flight; resting voltage when the battery was first used
Counting	energy drawn in this flight and on this battery; time since the battery was first used
Context	which flight this is on the battery; motors on or off

The two sequence models (LSTM and GRU) also get the last 60 seconds of voltage, current, power, energy drawn and motors-on, at 2 readings per second (120 steps).

Causality. Every input uses only data up to the present moment. We test this: change every reading after time t in a real flight, and check that no feature at or before t changes. Two early versions failed (one looked a few seconds ahead, one half a second) and were fixed.

A6. Model A: the five-algorithm comparison (Phase 3)

The baselines: what machine learning has to beat

Baseline	How it works
Fixed capacity	Every battery gives the same total. Subtract what has been used. (Reference only: 13.6 Wh error)
Energy counting	Read the starting charge from the resting voltage when the battery was inserted, then subtract energy used. This is what a battery management system does
Voltage lookup	Re-read the charge from the latest resting voltage before this flight, then subtract this flight's energy. The strongest non-ML method

A fair baseline matters. Against "fixed capacity" every model looks excellent. Against "voltage lookup" most of them do not.

The five algorithms, in plain words

Algorithm	In one line	Setting chosen
Linear Regression (ridge)	A weighted sum of the 25 features	regularisation 10
Random Forest	The average of many decision trees, each built on a random half of the data	depth 12, at least 20 rows per leaf
XGBoost	Trees built one after another, each correcting the errors of the ones before	400 shallow trees (depth 3), slow learning rate 0.02
LSTM	A neural network that reads the last minute of readings in order and keeps a memory	2 layers, 64 units
GRU	Like an LSTM with a simpler memory	2 layers, 64 units

Inside the GRU model, step by step

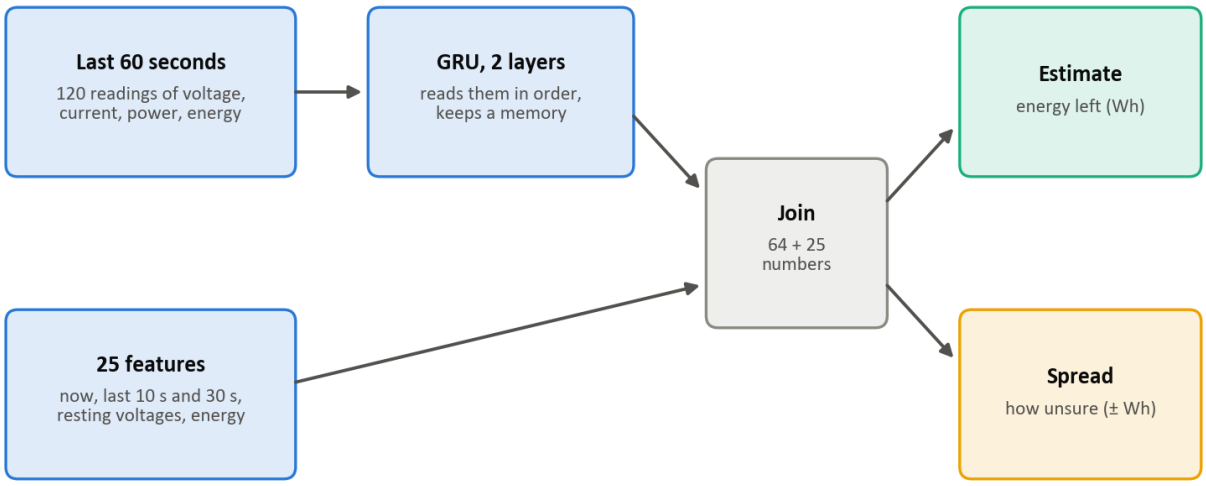


Figure 5. Model A. The spread box belongs to the final model only (section A8).

1. **Take the last 60 seconds:** 120 readings, two per second. Each reading is five numbers: voltage, current, power, energy drawn so far, and whether the motors are on.
2. **Put the numbers on a common scale,** using averages from the training batteries only.
3. **Read the readings in order.** The GRU keeps a memory of 64 numbers. At each reading it decides how much of the old memory to keep and how much to replace with what it has just seen. Two such layers are stacked.
4. **Keep the memory after the last reading.** It is a 64-number summary of how the battery behaved over the last minute.
5. **Join** that summary with the 25 features: 89 numbers.
6. **A small final layer** (89 → 64 → 1) turns them into one number: the energy left, in Wh.

An LSTM is built the same way, with an LSTM in place of the GRU in step 3. One network has about 45,000 adjustable weights, which is small.

Training. The network starts with random weights and is shown the training readings again and again (at most 60 passes). After each small batch its weights are nudged to reduce the error. The error measure is the Huber loss, which punishes large errors less harshly than squared error. Training stops early when the error on 15% of the *training* batteries stops improving.

How settings were chosen. A random search for each algorithm (10 settings for Random Forest, 30 for XGBoost, 8 for each neural network), scored by cross-validation grouped by battery.

Development results (cross-validation)

Model	Error (Wh)	Compared with voltage lookup
GRU	2.61	clearly better
LSTM	2.65	clearly better
Random Forest	3.02	within noise
Linear Regression	3.02	within noise
XGBoost	3.26	within noise
Voltage lookup	*3.22*	—
Energy counting	*4.49*	clearly worse

"Clearly" means the 95% interval of the difference excludes zero. The interval comes from resampling whole batteries 2,000 times (a bootstrap).

Why the sequence models win. They see how the voltage *responds* to load over the last minute, which reflects the health of that particular battery. A table of averages cannot show that as well.

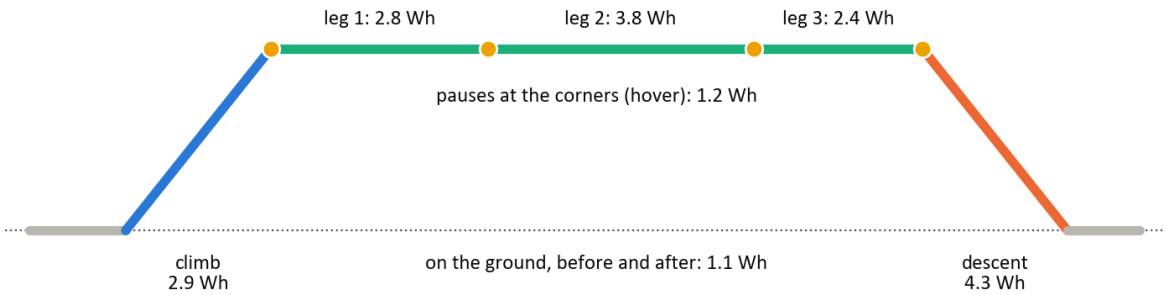
Why XGBoost does not beat the baseline. With only 49 development batteries, tree models can latch onto patterns of particular batteries that do not carry over to new ones. Splitting by battery exposes this. Splitting rows at random would have hidden it and made XGBoost look good.

Speed and size (laptop processor, one thread): every model predicts one reading in 6 ms or less. GRU about 4.8 ms and 0.18 MB; LSTM about 1.0 ms; Random Forest about 5.6 ms and 14.8 MB. Timings vary 3 to 4 times between battery and mains power, so they are quoted from a run on mains.

A7. Model B: energy a mission needs (Phase 4)

The structure.

Mission energy = climb + Σ over legs (power × time) + hover between legs + descent + ground



climb 2.9 + legs 9.1 + hover 1.2 + descent 4.3 + ground 1.1 = 18.6 Wh

90% range: 17.7 to 19.5 Wh

Figure 6. Model B adds up the parts of a mission. Example: a loop like route R1 (legs of 140, 198 and 119 m) at 8 m/s, 50 m altitude, with 250 g.

One leg, worked through. Leg 2 is 198 m at 8 m/s. Time = 198 ÷ 8 = 24.8 s, plus 2.4 s for speeding up and braking = 27.1 s. Power = 510 W. Energy = 510 × 27.1 ÷ 3600 = 3.8 Wh.

A "leg" is one straight stretch of the route. Every recorded flight flies the same loop, so the only way to learn the effect of distance is leg by leg: 587 legs from 193 usable flights.

Part	What is predicted	Inputs (all known before take-off)
Leg power	watts	speed, payload, altitude, wind
Leg time	seconds beyond distance ÷ speed	speed, leg distance, payload, wind
Climb	Wh	altitude, payload, wind
Descent	Wh	altitude, payload, wind
Hover	Wh	number of legs, payload, wind
Ground	a fixed 1.1 Wh (the typical value)	—

Why leg time is not just distance ÷ speed. The drone has to speed up and brake on every leg. That costs about 0.5 s at 4 m/s and 4.4 s at 12 m/s. The physics model for it is "speed ÷ acceleration + a constant", with a fitted acceleration of about 2.1 m/s².

The physics for power. Rotor momentum theory: power = a constant part + the power to hold the weight up (which depends on mass and forward speed) + air drag (which grows with speed cubed). Four numbers are fitted. The airframe mass is limited to the Matrice 100's real range (2.4 to 3.6 kg), because left free the fit gave an impossible 5 kg.

What was compared for each part. Physics alone, Linear Regression, Random Forest, XGBoost, and a hybrid: physics first, then XGBoost learns only the physics model's remaining error.

LSTM and GRU are not used for Model B. A mission plan is five or six numbers, not a sequence over time. The brief's five-algorithm comparison is done on Model A.

The main Model B: "physics-first"

Use a physics-shaped part (physics, or physics + XGBoost) unless a pure machine-learning part is more than 5% better.

- **What it ended up using:** physics + XGBoost for leg power, leg time and descent; physics alone for climb and hover.
- **Why not simply "the best part each"?** That option chose Linear Regression for leg power. A straight line cannot follow the real power curve, which is flat from 4 to 10 m/s and then rises. The result was an under-prediction of 0.44 Wh at 12 m/s. Under-predicting the energy a mission needs is the unsafe direction, and it was hidden inside a good overall error. Physics-first has the same overall accuracy with half that bias.

Development results

Model	Error
Physics-first (main)	**2.35%** (0.49 Wh)
XGBoost	2.47%
Physics alone	2.68%
Linear Regression	2.84%
Random Forest	3.42%
Guess the average flight	*19.6%*

The important test: conditions never seen. Hold out one whole altitude, or one whole payload, and predict it.

Model	Unseen altitude	Unseen payload
Random Forest, XGBoost	14%	9 to 10%
Physics-first	2.5%	3.1%

Trees can only repeat values they have seen. Physics-shaped models extend sensibly. This is the main argument for physics-first.

Wind barely matters here. The model is just as accurate with no wind input at all. Only wind speed is known, not its direction along the route, and on a closed loop head and tail winds cancel.

Warnings. The model reports when a mission is outside what it was tested on: speed 4 to 12 m/s, payload 0 to 750 g, altitude 25 to 100 m, wind 1.3 to 6.7 m/s, leg length 12 to 334 m.

A8. Honest ranges (Phase 5)

What "honest" means. A "90% range" should contain the true value about 90% of the time on data the model has not seen. The technical word is *calibrated*.

Model A's range

1. **Five GRUs, not one.** Each is trained from a different random start and holds out a different 15% of batteries. Each gives an estimate and its own spread. Where the five disagree, the model is unsure.
2. **The estimate and the spread are trained separately.** The textbook method trains them together. That made the estimate worse (2.51 against 2.39 Wh), so the spread part learns the size of the errors without being allowed to change the estimate.
3. **Conformal calibration.** On batteries the model never saw, measure how many "spreads" away the truth really fell. Use those measured values to set the range, in place of the textbook bell-curve values.

- Example: a bell curve says the 5% point is 1.64 spreads below the estimate. Our data says 2.08. The model's over-estimates are larger than a bell curve assumes, and the calibrated range allows for that.

4. **Every battery counts equally.** Thousands of readings from one battery share the same error, so they must not count as thousands of independent facts.
5. **No pre-flight reading, no answer.** Three development chains start with the motors already running, so the battery's starting charge is unknown. Errors there are 10 to 20 Wh. The model declines to answer, and the decision is NO-GO.

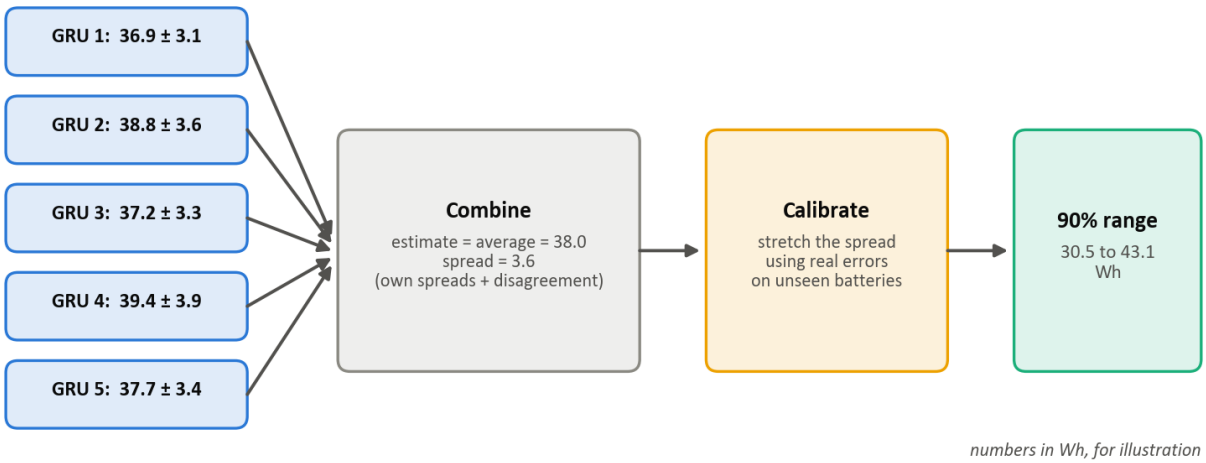


Figure 7. From five networks to one range.

The picture, worked through (illustrative numbers):

- **Estimate:** the average of the five, 38.0 Wh.
- **Spread:** 3.6 Wh. It combines each network's own spread with how much the five disagree.
- **Calibration:** on unseen batteries, the 5% point was 2.08 spreads below the estimate and the 95% point 1.43 spreads above.
- **Range:** 38.0 − 2.08 × 3.6 = 30.5 Wh, up to 38.0 + 1.43 × 3.6 = 43.1 Wh.
- The range reaches further down than up, because the model's over-estimates were larger than its under-estimates.

How the calibration numbers are found:

1. Predict every development battery with a model that never saw it (cross-validation).
2. For each reading, work out how many spreads off the truth was: (truth − estimate) ÷ spread.
3. Give every battery the same total weight.
4. For each of 23 probability levels (1% to 99%), find the value that this share of the results falls below. Those 23 values are the multipliers.
5. For a new reading: range point = estimate + spread × multiplier.

Model B's range

On unseen flights we know how wrong each part was (climb, descent, hover, ground in Wh; legs as a percentage). For a new mission, every unseen flight's set of errors is replayed on the new mission's parts, and the spread of the totals is the range.

- **The errors of one flight are kept together,** because they are linked: energy counted as "descent" on one flight is energy not counted as "ground" (correlation −0.63). Treating them as independent would overstate the uncertainty.
- **The leg error is a percentage,** so a mission with more or longer legs gets a wider range.
- **Several sorties on one battery share one error.** Flights flown back to back on a battery have errors that move together (about 46% of the error is shared). Treating them as independent gave "90%" ranges that held the truth only 81 to 85% of the time. A shared error gives 94 to 98%: cautious, which is the right side to err on.

Worked through, for the example mission of section A7 (18.6 Wh):

1. Take one unseen flight's errors, for example climb +0.2 Wh, descent +0.2 Wh, ground −0.4 Wh, legs +2%.
2. Apply them to the new mission's parts and add up. That gives one possible total.
3. Repeat for all 153 unseen flights: 153 possible totals.
4. The middle 90% of those totals is the range: 17.7 to 19.5 Wh.

Development results

	90% range held the truth	Width
Model A	89.5%	11.2 Wh
Model B	91.5%	2.1 Wh

A key finding. Model A's error is mostly a fixed offset per battery: about 3.3 Wh between batteries against 1.6 Wh within a battery. So the risk is per battery, not per moment. If the model over-estimates a battery, it does so for that battery's whole life.

A9. The decision and how it was tested (Phase 6)

P(success). The chance that energy available ≥ energy required, worked out from the two ranges. It is computed by a fixed calculation, not by random sampling, so the same inputs always give the same answer.

Rule. GO if P(success) ≥ 0.95.

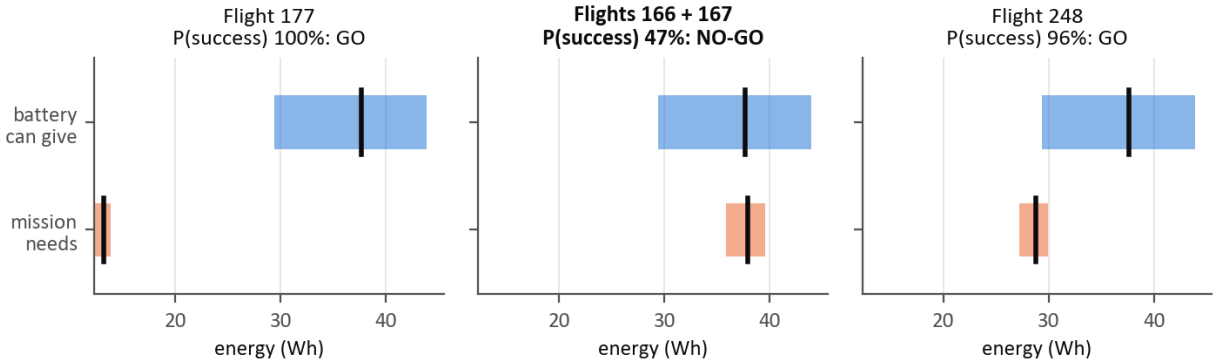


Figure 8. One battery, three missions (the worked example of section A12). Bars are 90% ranges; the black line is the best estimate.

Reading the picture. On the left, the mission's bar is far below the battery's bar, so success is almost certain. In the middle the two overlap, so it is close to a coin toss. On the right the bars only just touch: 96%, which passes.

How the number is calculated:

1. Take 400 equally likely values of energy available from Model A's range.
2. For each one, read from Model B's range the chance that the mission needs no more than that.
3. Average the 400 chances. That average is P(success).

Testing decisions without flying

We cannot fly new missions. But for recorded data we can ask: *would this battery, at this moment, have managed that recorded mission?*

- **A battery state** is a real reading. Its true remaining energy is known.
- **A mission** is a real flight, or two or three flights that really were flown on one battery. Its true energy is known.
- **The truth:** the mission would have succeeded if true energy available ≥ true energy required.

Pair states with missions (20,000 pairs), ask each rule for its decision, and count mistakes.

The four rules compared

	Rule	Approves when
P1	Minutes left (the brief)	minutes left ≥ mission duration
P2	Energy, best estimates	estimated energy available ≥ estimated energy needed
P3	EnduroSense	P(success) ≥ τ
P4	Oracle	the truth says yes (the best any rule could do)

P1 is given the mission's *true* duration, which favours it.

The two kinds of mistake

- **Unsafe approval:** approving a mission that would have cut into the reserve. Counted as a share of the missions that would fail.
- **Wasted refusal:** refusing a mission that would have been fine. Counted as a share of the missions that would succeed.



Figure 9. The four possible outcomes of a decision.

Every rule trades one against the other. Being stricter means fewer unsafe approvals and more wasted refusals. To compare fairly, P1 and P2 are also given a safety margin, chosen on development data, that makes them as safe as EnduroSense.

The fleet simulation

Four drones, 40 tasks a day, 200 days. Each battery is a real recorded chain and each task is a real recorded flight or pair of sorties. Only the pairing is simulated. A task goes to the approved drone with

the least energy (keeping fuller batteries for bigger tasks). If no drone is approved, the emptiest one swaps its battery.

A10. The final evaluation (Phase 7)

Order of events.

1. Rebuilt everything from the raw data with one command. It matched the stored results exactly.
2. Wrote down the four success criteria and how a near-miss would be reported.
3. Rehearsed the evaluation code on development data, to find coding errors without touching the test set.
4. Opened the test set and scored everything, with nothing adjusted.

The test set. 16 chains, 40 flights. 11 of the chains have an energy label, so Model A is judged on those 11 batteries (5,708 readings). Model B is judged on all 40 flights. Seven of the flights are on routes that do not exist in the development data (R2, R3, and R6, the longest).

The four criteria

#	Criterion	Result	Evidence
1	Model A beats both non-ML baselines	**Met**	2.19 Wh against 2.90 Wh and 4.21 Wh
2	Model B within 5%, including unseen routes	**Not met**	3.1% overall, but 5.7% on the 7 unseen-route flights
3	90% ranges hold the truth 88 to 92% of the time	**Not met, within sampling noise**	Model A 93.8%, Model B 80.0%
4	Better decisions than minutes-left and best estimates	**Met**	Unsafe approvals 0.61% against 12.4% and 3.6%

Model A on the test set

Model	Development	Test
Five GRUs averaged (final)	2.37 Wh	**2.19 Wh**
LSTM	2.45	2.32
GRU (single)	2.39	2.81
Random Forest	2.72	2.71
Linear Regression	2.64	2.63
XGBoost	2.91	2.75
Voltage lookup	2.82	2.90
Energy counting	3.98	4.21

(The development column here is for readings that have a pre-flight voltage, like every test reading. That is why it is lower than the 2.61 in section A6, which is over all readings.)

- **The ensemble is the only model clearly better than the baseline** on the test: −0.71 Wh, interval −1.33 to −0.12. It was better on 9 of the 11 batteries.
- **A single GRU was not.** It was best in development but scored 2.81 Wh on the test. Averaging five is what made the result dependable.
- **Its ranges:** 93.8% held the truth. Nine batteries were inside their range for at least 99.8% of their readings. Two were over-estimated (chain 70 by 7.4 Wh on average, chain 86 by 5.1 Wh).

Model B on the test set

Flights	n	Error
All test flights	40	3.1%
The route it was trained on (R1)	33	2.6%
Unseen routes (R2, R3, R6)	7	5.7%

- **All 7 unseen-route flights were over-predicted,** which is the safe direction. The excess is in the cruise legs. R6's cruise distance is about 820 m against 443 to 568 m in development.
- **Its ranges were too narrow:** they held the truth 80% of the time. Four of the six misses on the trained route are at 100 m altitude, a weak spot we had already flagged in development.

Decisions on the test set

Rule	Unsafe approvals	Wasted refusals	Unsafe approvals before take-off
P1 minutes left	12.4%	3.3%	12.8%
P2 best estimates	3.6%	3.9%	4.3%
P2 + margin from development (7.0 Wh)	0.55%	24.4%	0.00%
P3 EnduroSense	**0.61%**	24.3%	**0.00%**

- **Against the brief's rule:** 1 in 8 failing missions approved, against 1 in 160.
- **All 71 of EnduroSense's unsafe approvals involve two batteries:** the same two that Model A over-estimated.
- **The price:** EnduroSense refuses 24% of missions that would have been fine. These are close calls (typical true margin 3.8 Wh).
- **Minutes-left cannot be rescued with a margin:** with a margin that makes it as safe, it refuses 91% of good missions.
- **Best estimates plus a fixed margin did about as well as the probability** over all cases. Before take-off the probability refused fewer good missions (25% against 32%).
- **Is P(success) honest?** Mostly. Missions rated 95 to 99% succeeded 94% of the time: slightly over-confident, because of the two over-estimated batteries.

Fleet simulation on 8 unseen batteries: 12.0 unsafe missions per 100 with minutes-left, 0.73 with EnduroSense (16 times fewer), for about 10% more battery swaps.

A11. How we know the results are right

Safeguard	What it does
Split by battery	No battery is in both training and test
Locked test set	Chosen before modelling; gated in code; every opening logged. The log has three entries: the evaluation, one rerun after a check that changed only reporting code (identical numbers), and the run of the guide's extensions (section A13)
Criteria written first	They could not be adjusted after seeing results
One-command rebuild	`python scripts/run_all.py --final`, about 30 minutes. A rebuild with every model retrained from scratch reproduced all 74 result files and 19 model files byte for byte
166 automatic tests	Including causality, the split, and the arithmetic of every table
Mutation testing	Break one line of code on purpose and check that a test notices. Done for the uncertainty, decision, final-evaluation, dashboard and extension code
Six verification passes	After each stage, re-examine the work as a sceptic and recompute key numbers with separately written code

What the verification passes found (know two or three of these well):

1. Flights sorted by text time: 7 days out of order. The split had to be redone, before any model existed.
2. Some summary figures included test data. No model had used it. Redone on development data.
3. The first Model B under-predicted fast missions. Replaced by physics-first.
4. Multi-sortie missions were treated as having independent errors. They do not. Corrected.
5. The results write-up compared test error with a development figure from different data. Corrected.
6. Statements in the report and slides were typed by hand. They now come from the result files.

A12. The dashboard, report and slides

Dashboard (`streamlit run app/streamlit_app.py`). Nothing is trained in it. It loads the saved models and results.

Page	What it shows
Three missions, one battery	The worked example below
Mission check	Pick an unseen battery and a moment; plan a mission or choose a recorded one; see both ranges, P(success), and each rule's decision
Fleet	The simulation, with adjustable settings
Fleet planner (extension)	The guide's extensions: the planner and the computing choice (section A13)
Results	The criteria and all headline results
How it works	The method in plain words

The worked example (know it cold). Battery chain 86, before take-off. The model says 38 Wh above reserve; 33 Wh were really there. Three recorded missions; the minutes-left rule says GO to all three.

Mission	Really needed	P(success)	EnduroSense	Outcome
Flight 177	14 Wh	100%	GO	Fine
Flights 166 + 167 back to back	38 Wh	47%	**NO-GO**	5 Wh into the reserve
Flight 248	28 Wh	96%	GO	Fine, 5 Wh to spare

The case was picked by a fixed rule from the test data, not by hand.

Report and slides. Both are generated by scripts from the result files, so their numbers cannot disagree with the results.

A13. The guide's extensions (Phase 9)

The guide asked for two additions after the final evaluation:

1. **Multi-drone energy-aware task scheduling:** battery reserves, deadlines, payload, travel distance, charging and priorities, together.
2. **Energy-aware communication and edge computing:** process a task's data on board, on an edge server, or on another drone.

The most important thing to say about them: both are **simulations built on top of the finished models**. The dataset has no deadlines, priorities, chargers, radios or servers.

Real	Simulated
The batteries: recorded chains, with the battery model's saved ranges	When tasks arrive, their priority and deadline
The missions: recorded flights, with measured energy, flight time, payload and distance	Each drone's payload limit, the chargers and spare packs
Model B's ranges for each mission, and P(success)	Every radio, computer and edge-server setting
Hover power, 539 W, measured on development flights	

How it was done honestly. Built and checked on development batteries only. The three success criteria were written down first (`docs/extensions_protocol.md`). Then one run on the 8 unseen batteries, logged like the final evaluation. No model or earlier result changed.

Extension 1: the fleet planner

The set-up. 4 drones with different payload limits (750, 500, 500, 250 g), 100 tasks in an 8-hour day, 200 simulated days. Each task has a priority:

Priority	Share	Weight	Must be done within
High	20%	3	30 min
Normal	50%	2	90 min
Low	30%	1	240 min

Packs are swapped at the depot (2 minutes) and recharged on 2 chargers at 100 W, with 3 packs per drone in circulation. There is more work than the chargers can supply, so about 22 tasks a day are missed by every policy, even the oracle. That is deliberate: it forces choices.

How the planner decides, step by step (every minute that a drone is free and tasks are waiting):

1. **List the allowed pairs** of free drone and waiting task. A pair is allowed only if the drone can carry the payload, the task can be finished before its deadline, and P(success) ≥ 0.95.
2. **Score each pair:** priority first, then the closest deadline, then the battery that fits best (least energy to spare, so fuller packs are kept for bigger tasks).
3. **Match all of them at once** with the Hungarian algorithm (`linear_sum_assignment`), so one drone does not grab a task another drone needed.
4. **Swap early:** a free drone that is too low for any waiting task it could carry swaps its pack, if a charged one is ready.
5. **Never start a task that will be late:** one that can no longer finish on time is dropped.

Results on the unseen batteries:

Policy	Weighted tasks on time	High priority on time	Unsafe per 100 flown
First come, minutes left (the brief)	67%	53%	11.1
First come, EnduroSense check	71%	62%	0.62
Planner, best estimates	84%	96%	4.46
Planner, EnduroSense	**84%**	**95%**	**0.61**
Planner, oracle (knows the truth)	84%	96%	0

- **Planning is what gets the urgent work done:** 95% of high-priority tasks on time against 62% for a first-come queue.

- **EnduroSense is what keeps it safe:** the planner with best estimates is just as quick, but has 4.46 unsafe missions per 100 against 0.61.
- **The planner is as safe as the first-come queue with the same check** (0.61 against 0.62), while finishing far more work on time.
- **The price:** about 37 battery swaps a day against 34.

All three criteria were met.

Extension 2: where to process the data

Each task carries a computing job, for example checking images of the delivery site. Three options:

Option	Fast?	What it costs this drone
On board	10 to 60 s	the computer (15 W), plus hovering if the result is needed on site
Edge server at the depot	8 times faster, but the data must be sent, and the server may be busy	the radio (1 W), plus hovering while waiting
Another drone waiting at the depot	same speed as on board, data must be sent	the radio; the other drone's battery pays for the computing

The key insight: hovering is the cost. Half the jobs need the result before the drone leaves the site, so it hovers while it waits. Hovering draws about 539 W (measured). 30 seconds of hovering costs 4.5 Wh; 30 seconds of computing costs 0.125 Wh, and 30 seconds of radio 0.008 Wh. So the energy of a choice is mostly **how long it makes the drone hover**.

The link model, simply: speed of the link = bandwidth × log2(1 + signal-to-noise). The signal gets weaker with distance from the depot, varies randomly (fading), and is sometimes blocked (10% of the time, cut to a tenth).

The energy-aware rule, step by step:

1. **Before take-off:** add to the mission's energy the computing energy in a bad case (weak, blocked link and a busy server), so P(success) allows for it.
2. **At the site:** measure the link and ask the server's queue.
3. **Pick** the option with the least energy for this drone that still meets the job's deadline (the fastest one if none does).

Results on the unseen batteries:

Rule	Computing energy per task	Jobs on time
Always on board	2.60 Wh	90.2%
Always offload	0.95 Wh	81.5%
Energy-aware	**2.05 Wh**	**95.6%**

- **The energy-aware rule** offloaded 62% of jobs, kept 37% on board, and handed about 1% to another drone.
- **"Always offload" looks cheap but is not:** the planner refuses many on-site jobs before take-off, because a bad link could mean minutes of hovering. Only about 57% of the priority-weighted work gets done on time.
- **It adapts sensibly** (the sensitivity sweep). With a weak link it processes about 61 to 66% of jobs on board. With a strong link it offloads about 69 to 83%. When the server is busy more often, it hands more jobs to another drone.

What not to over-claim about the extensions

- **Simulated schedules.** The settings (priorities, deadlines, chargers, radio, server) are assumptions, not measurements.
- **Eight unseen batteries.** The ± values cover only the simulated days, not the small number of batteries.
- **Not built:** re-planning during a flight, wind direction, moving edge servers, and real radio measurements.

Part B. Numbers to know by heart

	Number
Flights / readings / battery chains	209 / 257,896 / 90
Test set	16 chains, 40 flights (11 chains with labels)
Battery pack / usable above reserve	99.9 Wh / about 65 Wh
Reserve / threshold	22.6 V at rest / 0.95
Unsafe approvals: minutes-left / best estimates / EnduroSense	**12.4% / 3.6% / 0.61%** (0.00% before take-off)
Good missions EnduroSense refuses	24%
Model A error	**2.19 Wh** (best non-ML method 2.90 Wh)
Model B error	**3.1%** (2.6% trained route, 5.7% unseen routes)
90% ranges held the truth	Model A 93.8%, Model B 80%
Fleet: unsafe missions per 100	12.0 against 0.73
Success criteria	2 of 4 met
Speed	6 ms or less per prediction on a laptop
Extensions (simulated, unseen batteries)	Planner: 95% of high-priority tasks on time against 62% first-come, 0.61 unsafe per 100. Computing: 95.6% of jobs on time at 2.05 Wh against 90.2% at 2.60 Wh on board
Rebuild time / tests	30 minutes / 166

Part C. Questions the panel may ask

Each has a short answer you can say aloud, then likely follow-ups.

C1. The idea

Q. In one minute, what did you build?

The brief asks for minutes of flight left. I predict two energies instead: what the battery can still give above a safe reserve, and what a planned mission will need. Each comes with a range that I checked on unseen data. From the two I get a probability of success, and a mission is approved only at 95% or more. On batteries and flights the models never saw, the brief's minutes-left rule approved 12.4% of missions that would have cut into the reserve. Mine approved 0.61%, and none at take-off.

- *Follow-up: did you still do what the brief asked?* Yes. Minutes left is predicted (0.8 min average error), and all five algorithms are compared with error, speed and size. The extension sits on top.
- *Follow-up: why did you change the brief?* I did not replace it; I added the step that turns the prediction into a decision. Our own results show minutes barely improve with a better battery model, because they depend on what the drone does next.

Q. Why is energy better than minutes?

Energy adds up and minutes do not. A mission's energy is the sum of its parts, and the battery loses exactly that amount. Minutes left depends on the power drawn right now, so a hovering drone looks as if it has plenty of time.

- *Follow-up: can you show that with your data?* Models with energy errors from 2.6 to 4.5 Wh all had minutes errors between 0.87 and 1.02 minutes. Improving the battery estimate hardly changes the minutes.

Q. What does "unsafe" mean? Does the drone crash?

No. It means the mission would have used more than the energy above the reserve, so it would have ended inside the safety buffer. The reserve exists so that this is not a crash.

Q. Why 22.6 V?

It is about 3.77 V per cell at rest, a conservative level, and enough recorded batteries reach it for the labels to exist (60 chains). It is a single setting, pending my mentor's confirmation.

- *Follow-up: what if we wanted 22.0 V?* Change one line and rerun, about 30 minutes. A lower reserve means more usable energy but fewer batteries in the data that reach it, so fewer labels.

Q. Why 95%?

It is a choice about how to trade safety against refused missions, and I measured the trade-off. At 90%: 1.07% unsafe approvals, 18% of good missions refused. At 95%: 0.61% and 24%. At 99%: 0.18% and 34%. The right value depends on what a reserve violation costs compared with a refused mission.

C2. The data

Q. Why this dataset?

It is real flight data with voltage and current recorded, so energy can be computed exactly, and it varies speed, payload and altitude in a planned grid. It is public, so the work can be checked.

- *Follow-up: what are its limits?* One drone, one battery type, 182 of 209 flights on one route, and flights of about 3 minutes. I say so in the report.

Q. The dataset does not say which battery was used. How do you know?

From the resting voltage. Between two flights on the same battery the resting voltage can only stay the same or recover slightly. So if the next flight starts within −0.10 to +0.40 V of where the last one ended, it is the same battery. A clear drop or a big jump means a swap.

- *Follow-up: how sure are you?* Six links are borderline and are flagged. I tried other windows and got 87 to 92 chains, close to my 90. And the reconstructed chains behave correctly: energy never decreases and resting voltage falls along every chain. Tests check both.
- *Follow-up: what if a link is wrong?* A missed link only makes the test harder for me, not easier, as long as both halves stay on the same side of the split. A false link would join two batteries; the label for that chain would be wrong. The asymmetric window is there to avoid that.

Q. Why split by battery and not randomly?

Readings from one battery are nearly copies of each other. With a random split the model is tested on batteries it has trained on, and the results look much better than they are. In my case XGBoost would have looked good. Split by battery, it does not beat the simple baseline.

Q. How did you choose the test set?

Before any modelling. I scored 5,000 random battery-level splits on how well the test part matched the whole dataset, kept the best, and forced the longest route into the test set as an unseen-distance check.

- *Follow-up: how do we know you did not peek?* The code refuses to load test rows outside the final evaluation, every opening is logged, the split file is hash-protected, and a verification pass searched every development file for test readings and found none.

Q. What data problems did you find?

Start times stored as text and sorted wrongly, which put 7 days out of order; 4 mis-recorded start times; a sign error in the documentation for vertical speed; altitude drifting up to 14 m after landing; and 76 sensor glitches.

- *Follow-up: how did you find the text-sorting bug?* In a verification pass. Battery chains looked broken on some days. The cause was "9:22" sorting after "10:05". Two of my own tests had the same bug and were passing because of it.
- *Follow-up: did you edit the raw data?* Never. Corrections are applied in code from a small file, each with voltage evidence.

Q. How do you compute energy?

Power is voltage times current. Energy is power times the time step, summed over the readings, about 8 per second.

C3. Model A

Q. What exactly does Model A predict, and where does the truth come from?

The watt-hours left above the reserve. For a battery flown down to the reserve, I know how much energy was drawn before its resting voltage reached 22.6 V. The label at any moment is that total minus the energy drawn so far.

- *Follow-up: you only have resting voltage between flights. How do you find the exact crossing?* By following the battery's discharge curve between the two nearest resting measurements. A straight line was biased by 3.3 Wh, because the curve flattens towards the reserve. The fitted curve has almost no bias.
- *Follow-up: how accurate is the truth itself?* About 1.5 to 2.2 Wh of noise. So an error of 2.2 Wh is close to the limit of what can be measured here.
- *Follow-up: was the curve fitted using test data?* No. Development batteries only.

Q. What are the inputs?

25 numbers describing the battery now and recently: voltage, current, power, their averages and trends over 10 and 30 seconds, internal resistance, the resting voltage before the flight and when the battery was first used, and energy drawn so far. The sequence models also read the last 60 seconds of readings.

- *Follow-up: could any of these leak the answer?* I test for that. I change every reading after a moment t and check no feature at or before t changes. Two early versions failed that test and were fixed.
- *Follow-up: isn't "energy drawn so far" almost the answer?* Only if every battery had the same capacity. They do not: full batteries give 47 to 77 Wh. Using energy drawn alone is the "fixed capacity" baseline, and it is off by 13.6 Wh.

Q. Explain the five algorithms simply.

Linear Regression is a weighted sum of the inputs. Random Forest averages many decision trees. XGBoost builds trees one after another, each fixing the previous errors. LSTM and GRU are neural networks that read readings in order and keep a memory; a GRU is a simpler LSTM.

- *Follow-up: what is the difference between LSTM and GRU?* An LSTM has a separate memory cell and three gates; a GRU merges them into two gates and has fewer parameters. Here they performed almost the same.

Q. Which is best, and why?

GRU and LSTM in development. On the test, the average of five GRUs was best at 2.19 Wh and is the only model clearly better than the baseline.

- *Follow-up: why do sequence models win?* They see how the voltage responds to changes in load over the last minute, which reflects that battery's health.
- *Follow-up: why does XGBoost do poorly? It usually wins on tables.* With 49 development batteries it learns patterns of individual batteries that do not transfer. The battery-level split exposes that.

- *Follow-up: your single GRU got worse on the test. Why?* One network is sensitive to its random start. In development it was 2.39 Wh, on the test 2.81. Averaging five removes most of that sensitivity. This is my evidence for using the ensemble.

Q. What is your baseline, and is it fair?

Voltage lookup: read the charge from the resting voltage before the flight, subtract the energy used in the flight. It is what you would do without machine learning. A naive baseline that assumes every battery is identical gives 13.6 Wh, and against that everything looks excellent, so I did not use it as the bar.

Q. Is 2.19 Wh good?

It is about 3% of the usable energy, it is 0.71 Wh better than the best non-ML method, and it is close to the noise in the labels themselves (1.5 to 2.2 Wh).

Q. How did you choose hyperparameters? Is there optimism in your numbers?

Random search scored by battery-grouped cross-validation. The same folds chose and scored the settings, so development numbers are slightly optimistic. That is why the test set exists; the test figures have no such optimism.

Q. Is it fast enough to run on a drone?

On a laptop processor, 6 ms or less per prediction for every model, and the GRU file is 0.18 MB. The ensemble is five GRUs, so roughly five times that. I have not timed it on on-board hardware; that is listed as a next step.

- *Follow-up: why is the LSTM faster than the GRU?* PyTorch's processor code for LSTMs is more heavily optimised. It is a library effect, not a property of the models.

C4. Model B

Q. How does Model B work?

It builds a mission from its parts: climb, each straight leg as power times time, hover between legs, descent, and ground. Each part is predicted from what is known before take-off: speed, payload, altitude, wind and leg distance.

- *Follow-up: why per leg?* Every recorded flight flies the same loop. Leg by leg is the only way to learn how distance matters: 587 legs of different lengths.
- *Follow-up: why not just distance divided by speed for the time?* The drone accelerates and brakes on every leg. That costs half a second at 4 m/s and over 4 seconds at 12 m/s.

Q. Why did you not use LSTM and GRU for Model B?

A mission plan is a handful of numbers, not a sequence over time. There is nothing for a sequence model to read.

Q. What is "physics-first"?

Use a physics-shaped model for each part, with XGBoost correcting its errors, unless a pure machine-learning part is more than 5% better.

- *Follow-up: why not just pick the most accurate model for each part?* I did at first. It chose a straight line for leg power, which under-predicted fast missions by 0.44 Wh. That is the unsafe direction, hidden inside a good average. Physics-first has the same accuracy with half that bias.
- *Follow-up: what is the evidence that physics helps?* Hold out an altitude or payload the model has never seen. Tree models are wrong by 9 to 14%. Physics-first stays at 2.5 to 3.1%.

Q. What physics?

For power, rotor momentum theory: a constant part, the power to hold the weight up, and drag that grows with speed cubed. For leg time, speed divided by an acceleration. For climb and descent, energy proportional to height and mass.

- *Follow-up: are the fitted physical numbers real?* No, and I say so. The fitted airframe mass sits at the top of the allowed range. The model is a physically shaped curve; its individual parameters should not be read as measurements.

Q. Model B missed its target on unseen routes. Explain.

5.7% against a 5% target, on 7 flights. All 7 were over-predicted, which is the safe direction. The excess is in the cruise legs. The longest route is about 820 m of cruise, against 443 to 568 m in development, so the model is being asked about something it never saw.

- *Follow-up: so the model is wrong?* It extrapolates less well than I wanted. I reported it as a miss and did not adjust the model, because adjusting it and scoring again on the same test data would be dishonest.
- *Follow-up: how would you fix it?* More routes of different lengths in the training data, and a wider range for missions outside the trained distances.

Q. Does wind matter?

Not in this data. The model is as accurate without the wind input. Only wind speed is known, not direction, and on a closed loop head and tail winds cancel. On a one-way leg in strong wind it would matter, and the model warns outside 1.3 to 6.7 m/s.

C5. Ranges and uncertainty

Q. What is a "calibrated range"?

A range that is as reliable as it claims. If I say 90%, the truth should be inside about 90% of the time on new data.

Q. How do you get the ranges?

By conformal calibration. I look at how wrong the model was on batteries it had not seen, and set the range from those real errors, in place of an assumed bell curve.

- *Follow-up: why not just use the neural network's own uncertainty?* It was too optimistic. Uncalibrated, the truth fell below the range 6.6% of the time against a nominal 5%. After calibration, 3.7%.
- *Follow-up: why five networks?* Where they disagree, the model is unsure. That disagreement, plus each network's own spread, gives a per-reading width which calibration then puts on the right scale.

- *Follow-up: why not dropout or a Bayesian network?* An ensemble plus conformal calibration is simple and makes no assumption about the shape of the errors. I did not compare against those alternatives.

Q. Why "every battery counts equally"?

Thousands of readings from one battery share the same error. Counting them as thousands of independent facts would make me far too confident. So each battery has the same total weight in the calibration.

Q. Your Model B range held the truth only 80% of the time. Isn't that a failure?

It missed the target band. With 40 flights from 16 batteries, 90% is still inside the sampling interval (65 to 92.5%), so I report it as "not met, within sampling noise". But the direction is unfavourable and I treat it as a real weakness: the ranges are too narrow on new data, mostly at 100 m altitude, which I had already flagged.

- *Follow-up: and Model A?* 93.8%, slightly wide of the band, with all the misses on two batteries.

Q. What happens when the model does not know the battery's starting charge?

It declines to answer, and the decision is NO-GO. Errors without a pre-flight reading were 10 to 20 Wh, too large to calibrate.

C6. The decision and its evaluation

Q. How do you compute P(success)?

I have a range for energy available and a range for energy required. P(success) is the chance that a value drawn from the first is at least a value drawn from the second. It is calculated on a fixed grid, so it is repeatable. I checked it against brute-force sampling: they agree within 0.0025.

- *Follow-up: you assume the two errors are independent. Are they?* Not quite. On the test they are positively linked (correlation 0.43). But Model B's error is about four times smaller than Model A's, so it hardly changes the result, and the link is in the safe direction.

Q. How can you evaluate decisions without flying?

By pairing a real battery state with a real recorded mission and asking whether that battery would have managed it. Both true energies were measured, so the right answer is known.

- *Follow-up: is that realistic? The battery and the mission are from different days.* It is a limitation and I state it. I also checked the real case: a battery before take-off against the flight it then actually flew. There the predicted margin's 90% range held the truth in all 30 test cases.

Q. What are your metrics?

Unsafe approvals (failing missions approved) and wasted refusals (good missions refused). I also report ranking quality across all thresholds (AUC: 0.994 for EnduroSense and best estimates, 0.975 for minutes-left).

Q. EnduroSense refuses 24% of good missions. Isn't that too cautious?

Those are close calls: their typical true margin was 3.8 Wh, and 93% had under 10 Wh to spare. The refusals come directly from the battery model's uncertainty, about 12 Wh wide. A better battery model would refuse fewer. And the threshold is a dial: at 90% it refuses 18%.

Q. Is the probability better than just adding a safety margin to the best estimates?

Over all cases, no measurable difference: 0.55% against 0.61% unsafe approvals at the same refusal rate. Before take-off the probability refused fewer good missions, 25% against 32%. The large gain is over minutes-left. I say this openly.

- *Follow-up: then why bother with the probability?* Two reasons. The margin had to be found by trial using known outcomes, and it differs by situation: 7.0 Wh overall, 11.4 Wh before take-off. The 95% threshold needed no tuning and is the same everywhere. And it adapts: it is stricter exactly when the model is less sure.

Q. Can't you just add a margin to minutes-left?

I tried. With a margin that makes it as safe, it refuses 91% of good missions.

Q. Tell me about the fleet simulation. What is real and what is simulated?

The batteries are real recorded chains and the tasks are real recorded flights. Only the pairing of tasks with batteries is simulated. With minutes-left, 12.0 missions per 100 cut into the reserve. With EnduroSense, 0.73, for about 10% more battery swaps.

C7. The results, honestly

Q. Two of your four criteria failed. Why should we trust this project?

Because I wrote the criteria down before opening the test set and reported the misses as misses. Model B was 5.7% on unseen routes against 5%, and the ranges were outside the 88 to 92% band. I did not adjust anything afterwards, because scoring an adjusted model on the same test data would give flattering numbers. The two criteria that matter most for the idea were met: the battery model beats the baselines, and the decisions are far safer than the brief's rule.

- *Follow-up: why not fix them and re-test?* The test set can only give an honest answer once. Any fix now has to be judged on new data.

Q. Your test set is small. How confident are you?

11 labelled batteries and 40 flights. Every interval I quote is wide because of that, and I state it. The headline comparison is still clear: minutes-left at 12.4% against 0.61% is not a difference that noise explains, and every resampling of the batteries shows the same direction.

Q. Unsafe approvals rose from 0.25% in development to 0.61% on the test. Why?

Two of the 11 test batteries were over-estimated by the battery model, and all 71 unsafe approvals come from those two. Development predicted exactly this kind of risk: the error is a fixed offset per battery.

Q. What is the biggest weakness?

The per-battery risk. If the model over-estimates one battery, it does so every time that battery is used. The fix is to let the model learn each battery's capacity as it flies. That is the first thing I would do next.

Q. What surprised you?

That a good non-ML baseline is hard to beat, that XGBoost did not beat it once I split by battery, and that the best single model in development was not the best on the test.

C8. Trust and engineering

Q. How do I know you did not tune on the test set?

The test set was chosen before modelling, the code refuses to read it outside the final evaluation, every opening is logged with a fingerprint of the code, and the criteria were written first. The log has three entries: the evaluation; one rerun after a verification pass that changed only how results are reported, with identical numbers; and the run of my guide's extensions, with its criteria written first. No model or earlier result changed.

Q. Is this reproducible?

One command rebuilds everything from the raw data in about 30 minutes. I rebuilt with every model retrained from scratch and got the same 74 result files and 19 model files, byte for byte.

- *Follow-up: neural networks are random. How can it be identical?* Every random seed is fixed, and one source of last-digit differences (a Random Forest adding its trees in varying order) was found and removed.

Q. What is mutation testing?

I break one line of code on purpose and check that a test fails. If no test notices, the tests have a gap. I did this for the uncertainty, decision, evaluation and dashboard code, and added tests until every planted bug was caught.

Q. What mistakes did you make?

Several, and I logged them. Sorting flights by text time. Summary figures that included test data, though no model used it. A first Model B that under-predicted fast flights. Treating multi-sortie errors as independent. And a results table that compared test error with a development figure from different data. Each was found in a verification pass and fixed.

C9. Practical scenarios

For each: what the system does today, and what would be needed. Say plainly when something is not built.

Q. A cold day. Batteries hold less. What happens?

Today: temperature is not in the dataset, so the model has never seen it directly. Two inputs react to it indirectly: internal resistance and voltage sag both rise in the cold. *Honestly:* the ranges were calibrated on this data's conditions, so in very different conditions they would no longer be reliable. *Needed:* cold-weather flights in the training data and a recalibration.

Q. The battery is two years old.

Today: the model sees a weaker battery through its voltage response and resting voltages, and batteries in the data already vary from 47 to 77 Wh. *The weakness:* the per-battery offset. *Needed:* update each battery's capacity estimate after every flight.

Q. The wind picks up mid-mission.

Today: Model B takes one expected wind speed for the mission, and warns above 6.7 m/s. Model A keeps reading the battery every second, so P(success) for the *rest* of the mission can be recomputed during the flight; the evaluation over all in-flight moments gave 0.61% unsafe approvals. *Not built:* automatic re-planning or a return-home trigger.

Q. A one-way delivery into a headwind.

Today: the model knows wind speed, not direction, and was trained on closed loops where head and tail winds cancel. It would under-estimate a long leg into the wind. *Needed:* wind direction relative to each leg as an input.

Q. The drone drops its payload halfway.

Built. A mission is a list of legs, each with its own payload. A delivery is "out with the payload, back empty".

Q. A mission longer or faster than anything in the data.

Today: the model answers but attaches a warning that the mission is outside its tested ranges. The test result shows why to take the warning seriously: on the longest route it was off by 5.9%. *Sensible policy:* treat a warned mission as NO-GO or require a larger margin.

Q. The voltage sensor glitches during a flight.

Today: in the offline data, glitches are replaced using the median of the 5 surrounding readings. That uses two readings after the glitch (a quarter of a second), so on a live drone the filter would need to use past readings only. A glitch also has limited effect because most inputs are 10 and 30 second averages.

Q. The battery was swapped while the drone was powered, so there is no pre-flight reading.

Built. The model declines to answer and the decision is NO-GO.

Q. A different drone, or a different battery.

Today: not tested. *What carries over:* the whole method (chains, labels, ranges, decision rule). *What must be redone:* both models retrained on that drone's flights, and the ranges recalibrated. Model B's physics has the right shape for any multirotor; its numbers would be refitted.

Q. How would this run on a real drone or ground station?

Model A needs only voltage and current, read once a second. Model B needs the mission plan. The decision is a fast calculation. Model A could run on board (small and quick); the scheduler would more naturally run on the ground. *Not done:* timing on on-board hardware.

Q. How would a scheduler use it for a fleet?

For each task, compute P(success) for each available drone. Give the task to an approved drone with the least energy, keeping fuller batteries for bigger tasks. If none is approved, swap a battery. That is what the simulation does.

Q. Several missions in a row on one battery.

Built. Their energies add, and they share one error, because back-to-back flights are wrong together. On the test these ranges held the truth 91% of the time.

Q. New flight data arrives next month. What do you do?

Add it, rerun the one-command pipeline, and recalibrate. If it is to serve as a new test, lock it away first.

Q. What if the model is confidently wrong?

That is the per-battery risk, and it happened on two test batteries. Three protections exist: the reserve itself (an unsafe approval ends inside the buffer; the typical shortfall was 2.2 Wh, the worst 7.3 Wh), the 95% threshold, and the range-of-validity warnings. The fourth, learning each battery over time, is not built.

Q. Who is responsible if it approves a mission that fails?

The system gives a probability with stated limits; it does not remove the need for a reserve or for operating rules. The threshold is a choice the operator makes knowing the measured trade-off.

C10. The extensions

Q. Your guide asked for scheduling and edge computing. What did you build?

Two simulations on top of the finished models. A fleet planner that assigns tasks considering battery reserves, deadlines, payload, distance, charging and priority. And a rule that decides where a task's data is processed: on board, on an edge server, or on another drone. The batteries and missions are real recordings; the schedule, the chargers and the radio settings are simulated.

- *Follow-up: why simulate?* The dataset has no deadlines, priorities, chargers or radio data. I said so up front, and kept these results separate from the main ones.
- *Follow-up: did you reopen the test set?* Yes, once, with the criteria written down first and the opening logged with a reason. No model or earlier result changed.

Q. Why does the planner beat a first-come queue?

A queue serves tasks in arrival order, so urgent tasks wait behind old ones and a heavy task can block it. The planner matches all waiting tasks to all free drones at once, urgent and high-priority first, and never starts a task it cannot finish on time. On unseen batteries: 95% of high-priority tasks on time against 62%.

- *Follow-up: what is the Hungarian algorithm?* It finds the best one-to-one matching between drones and tasks for a table of scores, in one step. Without it, the first drone could take a task that only the second drone could have carried.
- *Follow-up: is it optimal over the whole day?* No. It is optimal at each moment, given what is waiting. It does not hold a drone back for a high-priority task that has not arrived yet.

Q. Where does EnduroSense come in?

In the safety check of every pair. The planner with best estimates is just as fast but has 4.46 unsafe missions per 100; with P(success) ≥ 0.95 it has 0.61. Speed comes from planning, safety from the probability.

Q. Why do all policies miss about 22 tasks a day?

On purpose: 100 tasks need more energy than two chargers can supply in 8 hours. Under that pressure the policies differ in which tasks they finish, and the planner finishes the important ones.

Q. Communication energy is tiny. Why does the choice matter?

Because of hovering. A drone that waits on site for a result hovers at about 540 W. 30 seconds of that is 4.5 Wh; the radio and computer use a fraction of a watt-hour. The energy-aware rule picks the option that makes the drone wait least while meeting the deadline: 95.6% of jobs on time at 2.05 Wh per task, against 90.2% at 2.60 Wh for always on board.

- *Follow-up: where do the radio numbers come from?* Typical published values, not measurements. I swept link quality and server load: with a weak link it moves work on board, with a strong link it offloads more, which is what it should do.
- *Follow-up: what about mobility?* The link weakens with distance from the depot; jobs are sent from the delivery site or, if not urgent, on the way back from about half that distance. A moving edge server or re-routing to get a better link is not built.

C11. Questions about you

Prepare your own answers to these. Only you can answer them.

- What part did you find hardest, and what did you learn from it?
- Which decision would you make differently now?
- What tools and help did you use, and what did you check yourself?
- If you had another month, what would you do first? (Suggested: per-battery learning, and more routes.)

Part D. What not to claim

Do not say	Say this
"The probability beats a safety margin"	"It matched a tuned margin overall, refused fewer good missions before take-off, and needs no tuning. The big gain is over minutes-left"
"Model B is accurate to 3%"	"3.1% on the test; 2.6% on the trained route and 5.7% on unseen routes"
"The ranges are calibrated"	"Model A's held the truth 93.8% of the time. Model B's were too narrow at 80%"
"GRU is the best algorithm"	"The average of five GRUs was best. A single GRU did not beat the baseline on the test"
"The test set was used once"	"It was evaluated once, rerun once after a check that changed only reporting (identical numbers), and used once more for the guide's extensions. All three are logged"
"It works for any drone"	"It was built and tested on one drone and one battery type"
"Unsafe means a crash"	"Unsafe means ending inside the safety reserve"
"The planner was tested on real schedules"	"The schedules are simulated; the batteries and missions in them are real recordings"
"Offloading saves energy"	"Waiting less saves energy: hovering is the cost. Offloading helps when the link is good and the server free"
"The physics parameters are measured values"	"The physics gives the right shape; its individual numbers should not be read as measurements"

Part E. A two-minute version, if you are short of time

The brief asks for the remaining flight time of a UAV battery. Minutes are a weak basis for a decision, because they depend on what the drone is doing right now. So I predict two energies: what the battery can still give above a safe reserve, and what a planned mission needs. I give each a range, checked on data the models never saw, and combine them into a probability of success. A mission is approved at 95% or more.

I used 209 real flights of a DJI Matrice 100. Nobody recorded which battery was used, so I rebuilt that from resting voltages, and split the data by battery, with 16 batteries locked away from the start.

I compared the five algorithms in the brief. GRU and LSTM were the most accurate; the final battery model is five GRUs averaged, with 2.2 Wh error on unseen batteries against 2.9 for the best method without machine learning. The mission model is physics corrected by XGBoost, with 3.1% error.

On the locked test data, the brief's minutes-left rule approved 12.4% of missions that would have cut into the reserve. Mine approved 0.61%, and none at take-off. The price is caution: it refuses about a quarter of missions that would have been fine.

I set four success criteria beforehand. Two were met. Two were not: the mission model was 5.7% on routes it had never seen, and its ranges were too narrow. I reported those as they are and did not adjust the models afterwards.

My guide then asked for two extensions, which I built as simulations on the real batteries and missions. A fleet planner that weighs priority, deadline, payload, charging and the probability of success: it got 95% of urgent tasks done on time against 62% for a first-come queue, while staying as safe. And a choice of where to process a task's data: on board, on an edge server or on another drone. The surprise there is that hovering while waiting costs far more energy than the radio, so choosing by energy and deadline met 96% of computing deadlines for less energy than always processing on board.